CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

UNIVERSITY INSTITUTE OF COMPUTING

CASE STUDY

Student_Management_System

OBJECT ORIENTED PROGARMMING

25CAH-201

SUBMITED BY:

Name : RAKSHAK **UID : 25BCD10102**

Branch : BCA (DS) **Section: 25BCD2(A)**

Semester : 3

SUBMITED TO:

Name : Mr.Jitendra Kumar **Desigination: Professor**

Sign:_____

1. Introduction

Managing student information is a common task in schools and colleges. Student details such as identification number, name, age, course, and marks must be recorded, retrieved, and updated accurately. Maintaining these records manually can be time-consuming and may lead to duplicate entries, difficulty finding records, or accidental errors.

This case study examines a simple Student Management System developed in C++. The program uses object-oriented programming (OOP) to organize student data and file handling to preserve records between program runs. It is a console-based educational project intended to demonstrate fundamental C++ concepts.

2. Problem Statement

A small educational institution needs a basic application to manage student records. The application should allow a user to add, display, search, update, and delete student details. Records should remain available after the application closes, without requiring a database.

3. Objectives

- Create a menu-driven C++ application for student record management.
- Represent each student using a class and an object.
- Apply encapsulation by keeping student data members private.
- Use a vector to store multiple student records while the program is running.
- Use file streams to load and save records in a text file.
- Prevent duplicate student IDs and validate basic inputs.
- Calculate a grade from the student's marks.

4. Scope of the Project

The system supports one user working locally through a console menu. It stores a student's ID, name, age, course, and marks. The available operations are adding a record, displaying all records, searching by ID, updating a record, deleting a record, and exiting the program. The system uses a plain text file named students.txt for persistence.

The project is suitable for learning and demonstration. It does not include authentication, multiple-user access, a graphical interface, network access, or database-level safeguards.

5. Tools and Technologies

Tool / Concept	Purpose
C++ (C++17)	Programming language used to implement the application.
VS Code	Editor used to write and manage source code.
MSYS2 g++	Compiler used to build the executable.
OOP classes and objects	Organize student data and management operations.
STL vector	Store a changeable collection of Student objects.
fstream	Read records from and write records to a file.
students.txt	Persist student records between program runs.

6. System Design

6.1 Student Class

The Student class represents one student. Its private data members are ID, name, age, course, and marks. Public methods provide controlled access to the ID, display the student's information, calculate a grade, and write the student's data to a file.

6.2 StudentManager Class

The StudentManager class maintains a vector of Student objects and coordinates the program's operations. It searches for IDs, loads records at startup, saves the full list after changes, validates user input, and displays the menu.

6.3 Program Workflow

1. The application starts and attempts to load records from students.txt.
2. The main menu is displayed.
3. The user selects an operation.
4. The system validates input and performs the requested action.
5. After an add, update, or delete operation, the current records are saved to the file.
6. The menu repeats until the user selects Exit.

7. File Storage Format

Each student is stored on one line in students.txt. The fields are separated by a vertical bar (|). For example:

```
101|Aman Sharma|19|BCA|85
102|Priya Singh|20|BCA Data Science|92
```

The program reads each line and separates the fields to rebuild the student objects. The name and course fields must not contain the separator character (|), because it is used to divide fields.

8. Main Features

Feature	Description
Add Student	Accepts a new student's details and rejects an ID already in use.
Display Students	Shows all stored records in a table-like console format.
Search Student	Finds a student using the unique ID.
Update Student	Replaces the details associated with an existing ID.
Delete Student	Removes a record from the collection and saves the updated list.
Grade Calculation	Assigns a grade according to the marks entered.
Persistent Storage	Loads existing records at startup and saves changes to students.txt.

9. Basic Validation and Assumptions

- Student IDs must be positive whole numbers and must be unique.
- Age must be between 1 and 120.
- Marks must be between 0 and 100.
- Name and course fields cannot be empty or contain the | separator.
- The application is expected to be run from the folder containing students.txt.
- The grading scale used in the program is illustrative and can be changed to match institutional rules.

10. Testing Plan

Test	Action	Expected Result
Add record	Enter a new unique ID and valid details.	Record is added and saved.
Duplicate ID	Try to add an existing ID.	System displays a duplicate-ID message.
Display	Select Display All Students.	All loaded records are shown.
Search	Search for an existing ID.	Matching student details are shown.
Missing search	Search for an unknown ID.	System displays Student not found.
Update	Edit details for an existing ID.	Updated details replace the old details.
Delete	Delete an existing ID.	Record is removed from the list and file.
Persistence	Close and restart the application.	Saved records are loaded again.

11. Complete Source Code

This section contains the complete C++17 implementation used for the case study. Save the source as main.cpp in the project folder. The program uses the Student class to represent one student and the StudentManager class to manage records and file storage. Compile it with a C++17-compatible compiler such as MSYS2 g++.

```
#include <iostream>
#include <fstream>
```

```
#include <sstream>
#include <vector>
#include <string>
#include <limits>
#include <iomanip>

using namespace std;

const string FILE_NAME = "students.txt";

class Student {
private:
    int id;
    string name;
    int age;
    string course;
    double marks;

public:
    Student(int i, string n, int a, string c, double m)
        : id(i), name(n), age(a), course(c), marks(m) {}

    int getId() const {
        return id;
    }

    string grade() const {
        if (marks >= 90) return "A+";
        if (marks >= 80) return "A";
        if (marks >= 70) return "B";
        if (marks >= 60) return "C";
        if (marks >= 50) return "D";
        return "F";
    }

    void display() const {
        cout << left
             << setw(8) << id
             << setw(22) << name
             << setw(8) << age
             << setw(18) << course
             << setw(10) << fixed << setprecision(2) << marks
             << grade() << '\n';
    }

    void save(ofstream& file) const {
        file << id << '|' << name << '|' << age << '|'
             << course << '|' << marks << '\n';
    }
};

class StudentManager {
private:
    vector<Student> students;

    int findIndex(int id) const {
        for (int i = 0; i < static_cast<int>(students.size()); i++) {
            if (students[i].getId() == id)
                return i;
        }
        return -1;
    }

    void saveAll() const {
        ofstream file(FILE_NAME);
        if (!file) {
            cout << "Error: Could not save the file.\n";
            return;
        }
        for (const Student& s : students)
            s.save(file);
    }
```

```
void loadAll() {
    ifstream file(FILE_NAME);
    string line;

    while (getline(file, line)) {
        stringstream ss(line);
        string idText, name, ageText, course, marksText;

        if (!getline(ss, idText, '|') ||
            !getline(ss, name, '|') ||
            !getline(ss, ageText, '|') ||
            !getline(ss, course, '|') ||
            !getline(ss, marksText))
            continue;

        try {
            int id = stoi(idText);
            int age = stoi(ageText);
            double marks = stod(marksText);

            if (id > 0 && age > 0 &&
                marks >= 0 && marks <= 100 &&
                findIndex(id) == -1) {
                students.emplace_back(id, name, age, course, marks);
            }
        } catch (...) {
            // Ignore malformed records.
        }
    }
}

static int readInt(const string& prompt) {
    int value;
    while (true) {
        cout << prompt;
        if (cin >> value) {
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            return value;
        }
        cout << "Please enter a valid whole number.\n";
        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
    }
}

static double readMarks() {
    double marks;
    while (true) {
        cout << "Enter marks (0-100): ";
        if (cin >> marks && marks >= 0 && marks <= 100) {
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            return marks;
        }
        cout << "Invalid marks. Enter a value from 0 to 100.\n";
        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
    }
}

static string readText(const string& prompt) {
    string value;
    while (true) {
        cout << prompt;
        getline(cin, value);
        if (!value.empty() && value.find('|') == string::npos)
            return value;
        cout << "Input cannot be empty or contain '|'.\n";
    }
}

void printHeader() const {
    cout << left
         << setw(8) << "ID"
         << setw(22) << "Name"
```

```
                << setw(8) << "Age"
                << setw(18) << "Course"
                << setw(10) << "Marks"
                << "Grade\n";
            cout << string(76, '-') << '\n';
        }

    public:
        StudentManager() {
            loadAll();
        }

        void addStudent() {
            cout << "\n--- Add Student ---\n";
            int id = readInt("Enter student ID: ");

            if (id <= 0) {
                cout << "ID must be positive.\n";
                return;
            }
            if (findIndex(id) != -1) {
                cout << "Student ID already exists.\n";
                return;
            }

            string name = readText("Enter name: ");
            int age = readInt("Enter age: ");
            if (age <= 0 || age > 120) {
                cout << "Please enter a valid age.\n";
                return;
            }

            string course = readText("Enter course: ");
            double marks = readMarks();

            students.emplace_back(id, name, age, course, marks);
            saveAll();
            cout << "Student added successfully!\n";
        }

        void displayStudents() const {
            cout << "\n--- All Students ---\n";
            if (students.empty()) {
                cout << "No student records found.\n";
                return;
            }
            printHeader();
            for (const Student& s : students)
                s.display();
        }

        void searchStudent() const {
            int id = readInt("Enter student ID to search: ");
            int index = findIndex(id);
            if (index == -1) {
                cout << "Student not found.\n";
                return;
            }
            printHeader();
            students[index].display();
        }

        void updateStudent() {
            int id = readInt("Enter student ID to update: ");
            int index = findIndex(id);
            if (index == -1) {
                cout << "Student not found.\n";
                return;
            }

            cout << "Enter the student's updated details:\n";
            string name = readText("Enter new name: ");
            int age = readInt("Enter new age: ");
            if (age <= 0 || age > 120) {
```

```
            cout << "Invalid age. Update cancelled.\n";
            return;
        }

        string course = readText("Enter new course: ");
        double marks = readMarks();

        students[index] = Student(id, name, age, course, marks);
        saveAll();
        cout << "Student updated successfully!\n";
    }

    void deleteStudent() {
        int id = readInt("Enter student ID to delete: ");
        int index = findIndex(id);
        if (index == -1) {
            cout << "Student not found.\n";
            return;
        }

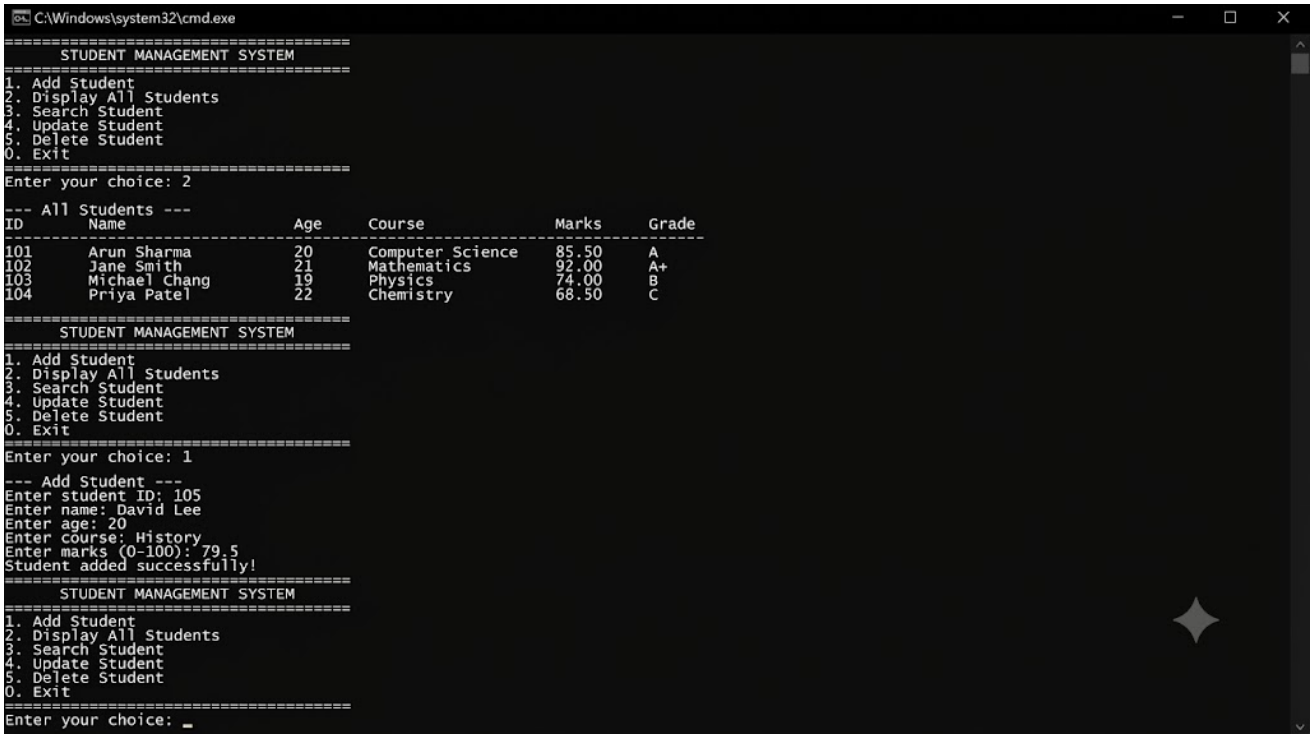
        students.erase(students.begin() + index);
        saveAll();
        cout << "Student deleted successfully!\n";
    }

    void run() {
        int choice;
        do {
            cout << "\n================================\n";
            cout << "   STUDENT MANAGEMENT SYSTEM\n";
            cout << "================================\n";
            cout << "1. Add Student\n";
            cout << "2. Display All Students\n";
            cout << "3. Search Student\n";
            cout << "4. Update Student\n";
            cout << "5. Delete Student\n";
            cout << "0. Exit\n";
            cout << "================================\n";

            choice = readInt("Enter your choice: ");
            switch (choice) {
                case 1: addStudent(); break;
                case 2: displayStudents(); break;
                case 3: searchStudent(); break;
                case 4: updateStudent(); break;
                case 5: deleteStudent(); break;
                case 0:
                    cout << "Thank you for using the system!\n";
                    break;
                default:
                    cout << "Invalid choice. Try again.\n";
            }
        } while (choice != 0);
    }
};

int main() {
    StudentManager manager;
    manager.run();
    return 0;
}
```

12. Sample Output



13. Expected Outcome

The expected result is a working console application that performs basic student record operations and retains valid records across program restarts. The project demonstrates how C++ classes, vectors, control flow, input validation, and file streams can be combined to solve a practical record-management problem.

14. Limitations and Future Improvements

A plain text file is simple to inspect and suitable for a small educational project, but it is not ideal for large datasets, concurrent users, or sensitive records. The current design rewrites the full file after each change and does not provide robust transaction recovery if a write is interrupted.

- Separate the code into header and implementation files.
- Add sorting, filtering, class averages, and highest-mark reports.
- Use a database such as SQLite for more structured storage.
- Add stronger file-error handling and safer update procedures.
- Add authentication and access control if the system is extended beyond a classroom demonstration.

15. Conclusion

The Student Management System is a practical introductory C++ mini-project. It demonstrates a menu-driven program structure, object-oriented design, data validation, and persistent file handling. By completing and testing the application, a student can gain experience in turning a simple real-world requirement into a working program.